

Module-4 Screenshots

```
fun main() {  
    val trafficLightColor = "Amber"  
  
    val message = when (trafficLightColor) {  
        "Red" -> "Stop"  
        "Yellow", "Amber" -> "Slow"  
        "Green" -> "Go"  
        else -> "Invalid traffic-light color"  
    }  
  
    println(message)  
}
```

Slow

2.4.20 ▾

JVM ▾

Program arguments



```
val lengthOfName = if (favoriteActor != null) {  
    favoriteActor.length  
} else {  
    0  
}  
  
println("Length using if/else: $lengthOfName")  
  
val lengthUsingElvis = favoriteActor?.length ?: 0  
  
println("Length using Elvis operator: $lengthUsingElvis")  
}
```

Name length: 9
The number of characters is 9.
Length using if/else: 9
Length using Elvis operator: 9

```

import kotlin.properties.ReadWriteProperty
import kotlin.reflect.KProperty

open class SmartDevice(val name: String, val category: String) {

    var deviceStatus = "online"
        protected set

    open val deviceType = "unknown"

    open fun turnOn() {
        deviceStatus = "on"
    }

    open fun turnOff() {
        deviceStatus = "off"
    }
}

class SmartTvDevice(deviceName: String, deviceCategory: String) :
    SmartDevice(name = deviceName, category = deviceCategory) {

    override val deviceType = "Smart TV"

```

```

        private var speakerVolume by RangeRegulator(initialValue = 2, minValue = 0, maxValue = 100)

        private var channelNumber by RangeRegulator(initialValue = 1, minValue = 0, maxValue = 200)

        fun increaseSpeakerVolume() {
            speakerVolume++
            println("Speaker volume increased to $speakerVolume.")
        }

        fun nextChannel() {
            channelNumber++
            println("Channel number increased to $channelNumber.")
        }

        override fun turnOn() {
            super.turnOn()
            println(
                "$name is turned on. Speaker volume is set to $speakerVolume and channel number is " +
                "set to $channelNumber."
            )
        }

        override fun turnOff() {
            super.turnOff()

```

```

        }
    }
}

class SmartLightDevice(deviceName: String, deviceCategory: String) :
    SmartDevice(name = deviceName, category = deviceCategory) {

    override val deviceType = "Smart Light"

    private var brightnessLevel by RangeRegulator(initialValue = 0, minValue = 0, maxValue = 100)

    fun increaseBrightness() {
        brightnessLevel++
        println("Brightness increased to $brightnessLevel.")
    }

    override fun turnOn() {
        super.turnOn()
        brightnessLevel = 2
        println("$name turned on. The brightness level is $brightnessLevel.")
    }

    override fun turnOff() {

```

```

        super.turnOff()
        brightnessLevel = 0
        println("Smart Light turned off")
    }
}

```

```

class SmartHome(
    val smartTvDevice: SmartTvDevice,
    val smartLightDevice: SmartLightDevice
) {

```

```

    var deviceTurnOnCount = 0
    private set

```

```

    fun turnOnTv() {
        deviceTurnOnCount++
        smartTvDevice.turnOn()
    }

```

```

    fun turnOffTv() {
        deviceTurnOnCount--
        smartTvDevice.turnOff()
    }

```

```

    fun turnOnLight() {

```

```

        fun increaseVolume() {
            smartTvDevice.increaseSpeakerVolume()
        }

```

```

        fun changeTvChannelToNext() {
            smartTvDevice.nextChannel()
        }

```

```

        fun turnOnLight() {
            deviceTurnOnCount++
            smartLightDevice.turnOn()
        }

```

```

        fun turnOffLight() {
            deviceTurnOnCount--
            smartLightDevice.turnOff()
        }

```

```

        fun increaseLightBrightness() {
            smartLightDevice.increaseBrightness()
        }

```

```

        fun turnOffALLDevices() {
            turnOffTv()

```

```

class RangeRegulator(
    initialValue: Int,
    private val minValue: Int,
    private val maxValue: Int
) : ReadWriteProperty<Any?, Int> {

    var fieldData = initialValue

    override fun getValue(thisRef: Any?, property: KProperty<*>): Int {
        return fieldData
    }

    override fun setValue(thisRef: Any?, property: KProperty<*>, value: Int) {
        if (value in minValue..maxValue) {
            fieldData = value
        }
    }
}

fun main() {
    var smartDevice: SmartDevice = SmartTvDevice("Android TV", "Entertainment")
    smartDevice.turnOn()
}

```

```

fun main() {
    var smartDevice: SmartDevice = SmartTvDevice("Android TV", "Entertainment")
    smartDevice.turnOn()

    smartDevice = SmartLightDevice("Google Light", "Utility")
    smartDevice.turnOn()
}

```

2.4.20 JVM Program arguments



[Copy link](#)

[Share code](#)

```

private val minValue: Int,
private val maxValue: Int
) : ReadWriteProperty<Any?, Int> {

    var fieldData = initialValue

    override fun getValue(thisRef: Any?, property: KProperty<*>): Int {
        return fieldData
    }

    override fun setValue(thisRef: Any?, property: KProperty<*>, value: Int) {
        if (value in minValue..maxValue) {
            fieldData = value
        }
    }
}

```

Android TV is turned on. Speaker volume is set to 2 and channel number is set to 1.
Google Light turned on. The brightness level is 2.

```

fun main() {
    val treatFunction = trickOrTreat(false) { "$it quarters" }
    val trickFunction = trickOrTreat(true, null)

    repeat(4) {
        treatFunction()
    }

    trickFunction()
}

fun trickOrTreat(
    isTrick: Boolean,
    extraTreat: ((Int) -> String)?
): () -> Unit {
    if (isTrick) {
        return {
            println("No treats!")
        }
    } else {
        return {
            val quantity = 5

            if (extraTreat != null) {
                println(extraTreat(quantity))
            } else {
                println("Have a treat!")
            }

            println("Have a treat!")
        }
    }
}

```

```
fun trickOrTreat(
    isTrick: Boolean,
    extraTreat: ((Int) -> String)?
): () -> Unit {

    if (isTrick) {
        return {
            println("No treats!")
        }
    } else {
        return {
            val quantity = 5

            if (extraTreat != null) {
```

```
5 quarters
Have a treat!
5 quarters
Have a treat!
5 quarters
Have a treat!
5 quarters
Have a treat!
No treats!
```

```

fun main() {
    val morningNotification = 51
    val eveningNotification = 135

    printNotificationSummary(morningNotification)
    printNotificationSummary(eveningNotification)
}

fun printNotificationSummary(numberOfMessages: Int) {
    if (numberOfMessages < 100) {
        println("You have ${numberOfMessages} notifications.")
    } else {
        println("Your phone is blowing up! You have 99+ notifications.")
    }
}

```

```

fun main() {
    val morningNotification = 51
    val eveningNotification = 135

    printNotificationSummary(morningNotification)
    printNotificationSummary(eveningNotification)
}

fun printNotificationSummary(numberOfMessages: Int) {
    if (numberOfMessages < 100) {
        println("You have ${numberOfMessages} notifications.")
    } else {
        println("Your phone is blowing up! You have 99+ notifications.")
    }
}

```

You have 51 notifications.
Your phone is blowing up! You have 99+ notifications.

```

fun main() {
    val child = 5
    val adult = 28
    val senior = 87

    val isMonday = true

    println("The movie ticket price for a person aged $child is \${ticketPrice(child, isMonday)}.")
    println("The movie ticket price for a person aged $adult is \${ticketPrice(adult, isMonday)}.")
    println("The movie ticket price for a person aged $senior is \${ticketPrice(senior, isMonday)}.")
}

```

```

fun ticketPrice(age: Int, isMonday: Boolean): Int {
    // ...
}

```

```

The movie ticket price for a person aged 5 is $15.
The movie ticket price for a person aged 28 is $25.
The movie ticket price for a person aged 87 is $20.

```

```

fun main() {
    printFinalTemperature(27.0, "Celsius", "Fahrenheit") { 9.0 / 5.0 * it + 32 }
    printFinalTemperature(350.0, "Kelvin", "Celsius") { it - 273.15 }
    printFinalTemperature(10.0, "Fahrenheit", "Kelvin") { 5.0 / 9.0 * (it - 32) + 273.15 }
}

```

```

fun printFinalTemperature(
    initialMeasurement: Double,
    initialUnit: String,
    finalUnit: String,
    conversionFormula: (Double) -> Double
) {
    // ...
}

```

```

27.0 degrees Celsius is 80.60 degrees Fahrenheit.
350.0 degrees Kelvin is 76.85 degrees Celsius.
10.0 degrees Fahrenheit is 260.93 degrees Kelvin.

```



```

fun main() {
    val brunoSong = Song("We Don't Talk About Bruno", "Encanto Cast", 2022, 1_000_000)
    brunoSong.printDescription()
    println(brunoSong.isPopular)
}

```

```

class Song(
    val title: String,
    val artist: String,
    val yearPublished: Int,
    val playCount: Int
)

```

```

We Don't Talk About Bruno, performed by Encanto Cast, was released in 2022.
true

```

2.4.20 ▾

JVM ▾

Program arguments



[Copy link](#)

```

fun main() {
    val amanda = Person("Amanda", 33, "play tennis", null)
    val atiqah = Person("Atiqah", 28, "climb", amanda)

    amanda.showProfile()
    atiqah.showProfile()
}

class Person(val name: String, val age: Int, val hobby: String?, val referrer: Person?) {
    fun showProfile() {
        println("Name: $name")
        println("Age: $age")
        println("Hobby: $hobby")
        if (referrer != null) {
            println("Referrer: $referrer")
        }
    }
}

```

```

Name: Amanda
Age: 33
Likes to play tennis. Doesn't have a referrer.

Name: Atiqah
Age: 28
Likes to climb. Has a referrer named Amanda, who likes to play tennis.

```

2.4.20 ▾

JVM ▾

Program arguments



Copy link



```
class FoldablePhone(var isFolded: Boolean = true): Phone() {  
    override fun switchOn() {  
        if (!isFolded) {  
            isScreenLightOn = true  
        }  
    }  
  
    fun fold() {  
        isFolded = true  
    }  
  
    fun unfold() {  
        isFolded = false  
    }  
}
```

The phone screen's light is off.
The phone screen's light is on.

```
fun main() {  
    val winningBid = Bid(5000, "Private Collector")  
  
    println("Item A is sold at ${auctionPrice(winningBid, 2000)}.")  
    println("Item B is sold at ${auctionPrice(null, 3000)}.")  
}
```

```
class Bid(val amount: Int, val bidder: String)
```

```
fun auctionPrice(bid: Bid?, minimumPrice: Int): Int {  
    return bid?.amount ?: minimumPrice  
}
```

Item A is sold at 5000.
Item B is sold at 3000.